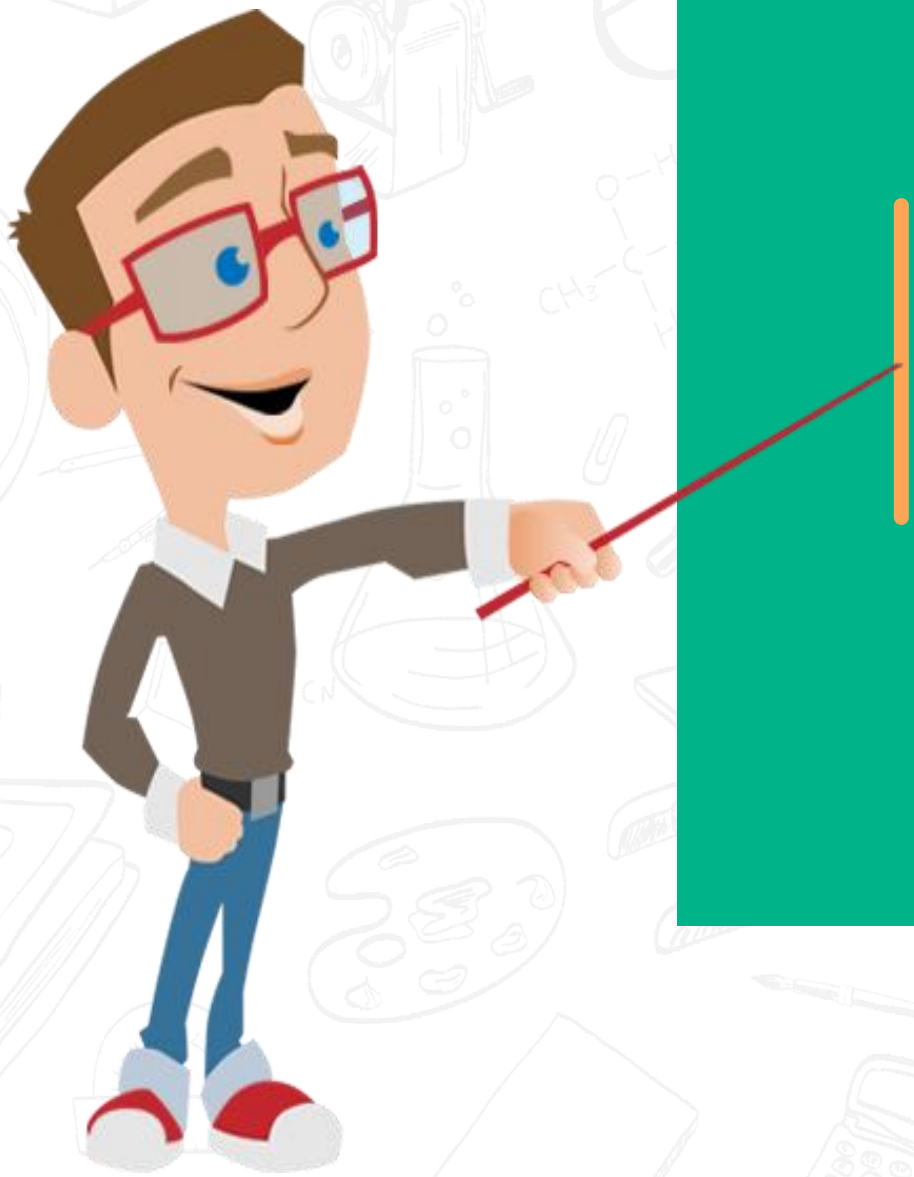


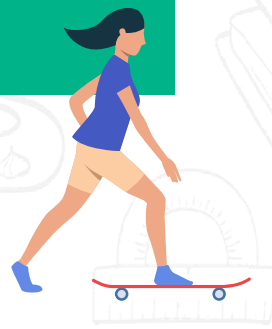
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 8-1

1 C++ basic

2 ~

3 ~

4 ~

Objectives

1、掌握 c++ 的基本语法

2、学会在 Linux 中运行 C/C++ 程序

3、了解 OPP 概念

4、学会建立 C++ 工程



Pre-assessment

1、是否在 Linux 中使用过C/C++?

2、是否用过 C++ 编写 ROS 程序?

3、是否了解面向对象编程的概念?



1

C++ basic

1.1 Getting started with C++

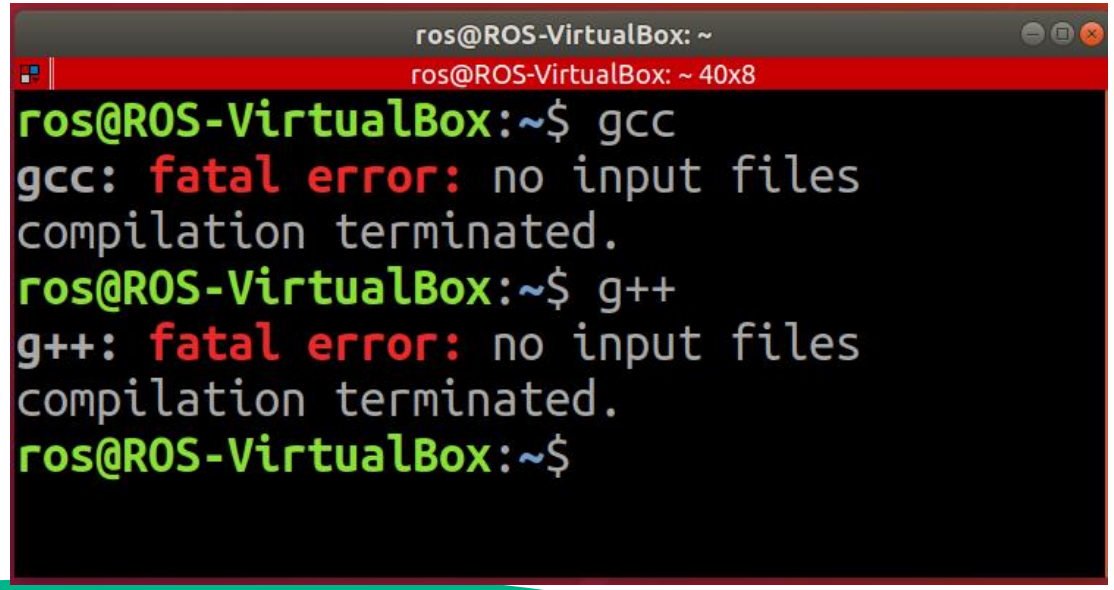
1.1.1 Overview

- C++ 是 C 语言的加强版，C++ 语言最初被称为“带类的C”，由计算机程序员 Bjarne Stroustrup 于1979年开发
- 和 C 一样，C++ 也是一种需要被编译的语言，它需要一个编译器将源代码转换为可执行代码
- Ubuntu自带一个内置的 C/C++编译器，称为 GCC/G++
- GCC 由 Richard Stallman 为 GNU 项目编写的（创立自由软件 GNU、Emacs 文字编辑器、GCC编译器、GDB调试器、GPL协议）
- GCC 代表 GNU 编译器的集合，其可以编译 C、C++、Objective-C、Fortran、Ada 和 Go 等多种语言，以及这些语言的库

1.1 Getting started with C++

1.1.2 在Ubuntu中运行C/C++

- Ubuntu 中预装了 C 和 C++ 编译器，其中 C 语言的编译器是 GCC，C++ 的编译器是 G++
- gcc 和 g++ 是两个编译器的 Shell 命令
- 在终端中运行上述命令，若出现下述错误提示，则说明系统已经预安装这些编译器，否则需要使用 apt 安装



1.1 Getting started with C++

1.1.2 在Ubuntu中运行C/C++

- 安装 C 和 C++ 编译器

```
$ sudo apt-get update
```

```
$ sudo apt-get install build-essential manpages-dev
```

```
$ sudo apt-get install gcc g++
```

1.1.2 在Ubuntu中运行C/C++

- 验证安装 C 和 C++ 编译器

```
$ whereis gcc
```

whereis 查看 gcc/g++ 命令及其使用说明页的路径

```
$ whereis g++
```

```
ros@ROS-VirtualBox:~$ whereis gcc
gcc: /usr/bin/gcc /usr/lib/gcc /usr/share/man/man1/gcc.1.g
z
ros@ROS-VirtualBox:~$ whereis g++
g++: /usr/bin/g++ /usr/share/man/man1/g++.1.gz
ros@ROS-VirtualBox:~$
```

```
$ which gcc
```

显示当前使用的编译器的系统路径

```
$ which g++
```

```
ros@ROS-VirtualBox:~$ which gcc
/usr/bin/gcc
ros@ROS-VirtualBox:~$ which g++
/usr/bin/g++
ros@ROS-VirtualBox:~$
```

1.1.2 在Ubuntu中运行C/C++

- 验证安装 C 和 C++ 编译器

```
$ gcc --version
```

显示编译器的版本号

```
$ g++ --version
```

```
ros@ROS-VirtualBox:~$ gcc --version
gcc (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditio
ns.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PA
RTICULAR PURPOSE.
```

```
ros@ROS-VirtualBox:~$ g++ --version
g++ (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
Copyright (C) 2017 Free Software Foundation, Inc.
This is free software; see the source for copying conditio
ns.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PA
RTICULAR PURPOSE.
```

1.1.2 在Ubuntu中运行C/C++

- GNU 项目调试器 GDB 简介
  - 调试器是一个可以运行和控制另一个程序的程序，通过检查每一行代码来检查程序的问题或 Bug
  - Ubuntu 自带一个名为 GNU Debugger 的调试器，简称 GDB(www.gnu.org/software/gdb/)

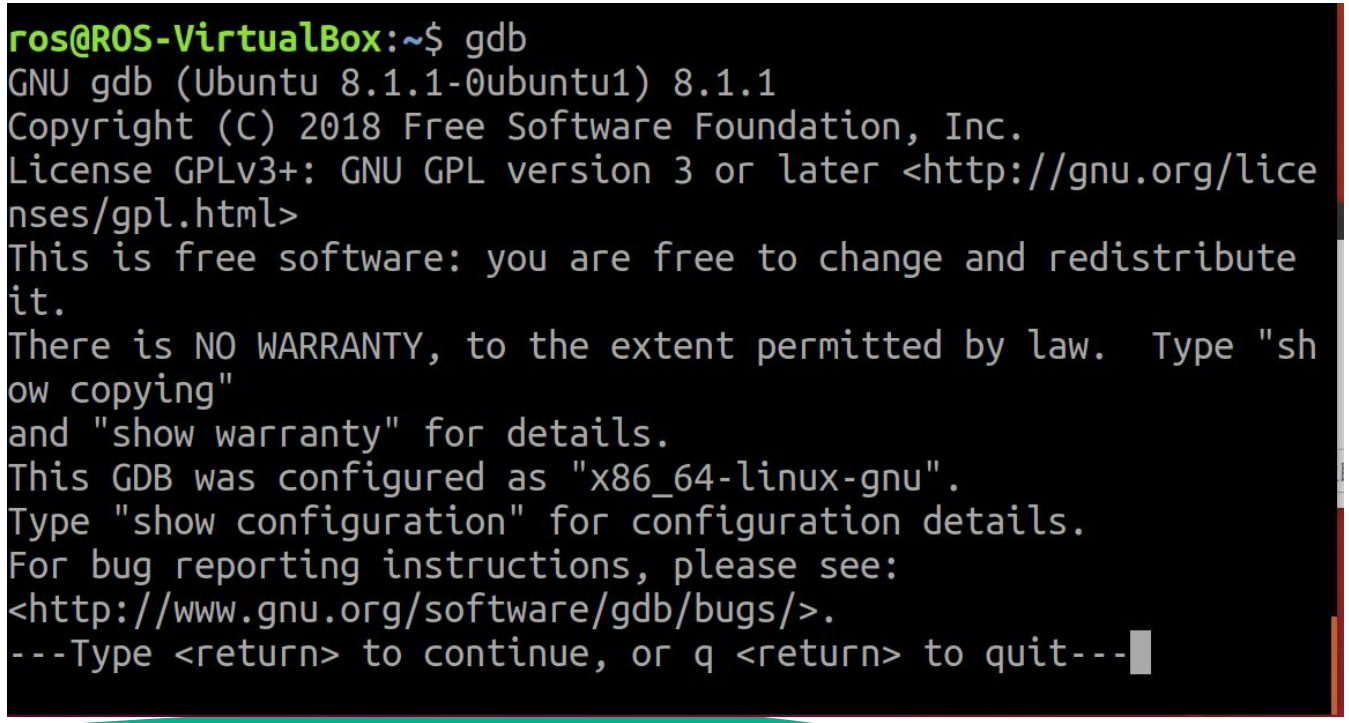
- 在 Ubuntu 中安装 GDB
  - 最新版本的 Ubuntu 已经安装了 GDB，若系统没有预安装，可以使用以下命令安装GDB

```
$ sudo apt-get install gdb
```

1.1.2 在Ubuntu中运行C/C++

- 验证安装GDB
  - 在终端中输入以下命令，验证GDB是否安装成功

```
$ gdb
```




```
$ gdb --version
```

查看 GDB 版本

1.1.2 在Ubuntu中运行C/C++

- 编写第一个C++程序
  - 在家目录中创建 C++_work 目录
  - 进入 C++_work 目录
  - 打开 gedit，键入以下代码：

```
#include <iostream>

using namespace std;
int main()
{
        cout<<"Hello ROS"<<endl;
        return 0;
}
```

  - 保存为hello_ROS.cpp

1.1.2 在Ubuntu中运行C/C++

- 代码解释
  - hello_ROS.cpp 将打印消息“Hello ROS”
  - #include <iostream>是 C++ 头文件，包含输入/输出函数，如从键盘输入或者打印消息到屏幕
  - 第二行使用 namespace std， 即使用命名空间
    - 命名空间是C++中的一个特性，用于对一组实体进行分组
    - iostream 中使用 std 定义命名空间
    - 当使用 namespace std 时，可以访问命名空间 std 中包含的所有函数或者其他实体，如 cout() 和 cin() 等函数
    - 若不使用这行代码，则必须在函数前面添加 std::，表示用于访问该命名空间中的函数，例如：std::cout 是打印一个消息的函数

1.1.2 在Ubuntu中运行C/C++

- 代码解释
  - 主函数中，使用 cout<<”Hello ROS”<<endl 打印一条消息
  - endl 表示在打印消息之后换行
  - 消息打印完毕后，函数返回0并退出程序

1.1.2 在Ubuntu中运行C/C++

- 编译代码
  - 打开一个新终端，将新终端的路径切换到 hello_ROS.cpp 所在目录
  - 在终端中键入以下命令进行编译：

```
$ g++ hello_ROS.cpp
```

  - G++ 编译器检查代码，若没有错误，其会创建一个名为 a.out 的可执行文件
  - 使用以下命令执行此文件：

```
$ ./a.out
```

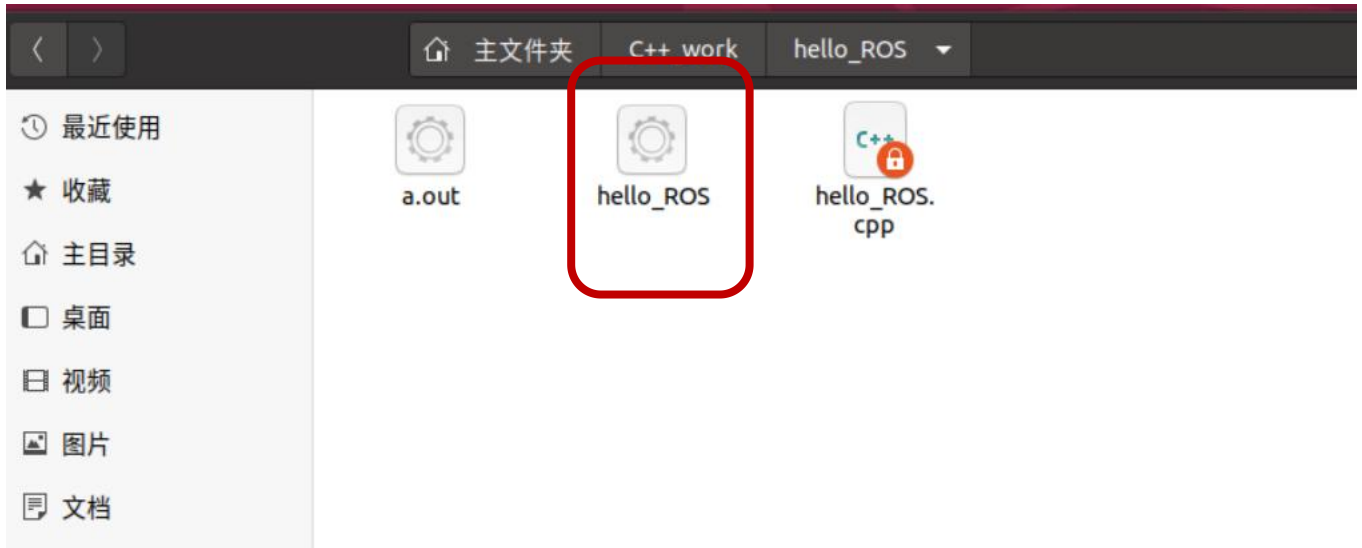
```
ros@ROS-VirtualBox:~/C++_work$ ./a.out
Hello ROS
```

1.1.2 在Ubuntu中运行C/C++

➢ 编译代码

- 若要创建具有特定名称的可执行文件，可使用如下命令：

```
$ g++ hello_ROS.cpp -o hello_ROS
```




```
$ ./hello_ROS
```

1.1.2 在Ubuntu中运行C/C++

- 代码调试
  - 使用调试器，可以遍历每一行代码并检查每个变量的值
  - 新建 sum.cpp，并键入以下代码：

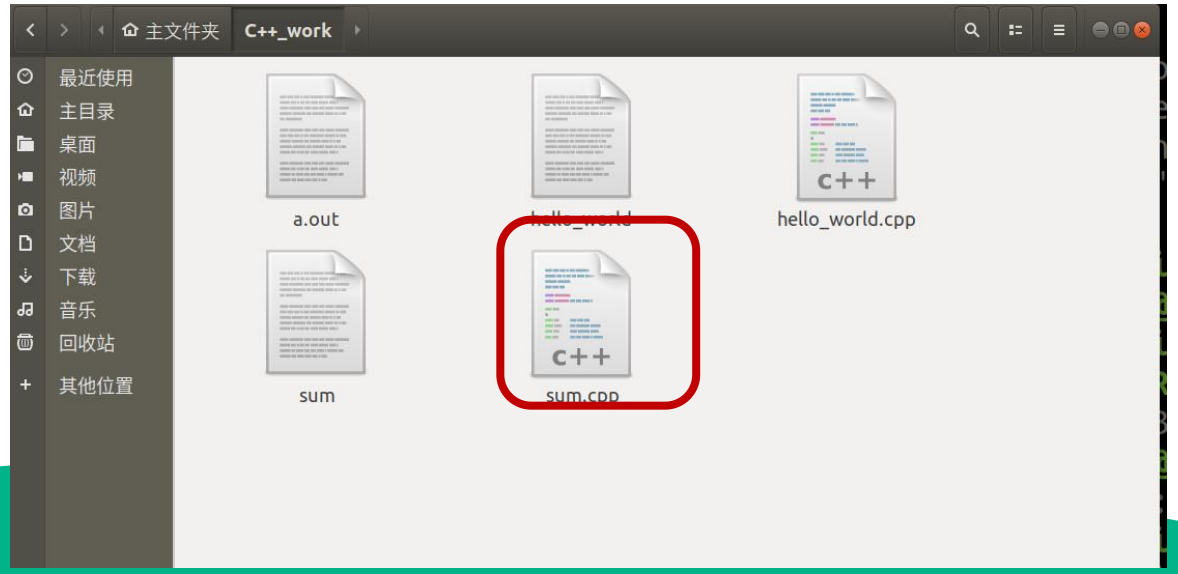
```
#include <iostream>

using namespace std;
int main()
{
        int num_1 = 3;
        int num_2 = 4;
        int sum = num_1 + num_2;
        cout<<"Sum: "<<sum<<endl;
        return 0;
}
```

1.1.2 在Ubuntu中运行C/C++

- 代码调试
  - 要调试/检查每一行代码，必须先使用 g++ -g 选项编译 sum.cpp 文件
  - 上述命令使用调试符号编译代码，并使其能够适用于 GDB 调试
  - 在终端中键入如下代码：

```
$ g++ -g sum.cpp -o sum
```



1.1.2 在Ubuntu中运行C/C++

- 代码调试
  - 在终端中运行以下命令：

```
$ ./sum
```

```
ros@ROS-VirtualBox:~/C++_work$ ./sum
Sum: 7
```

  - 创建可执行文件后，通过以下命令调试可执行文件：

```
$ gdb sum
```

1.1.2 在Ubuntu中运行C/C++

- 代码调试

  - sum 是可执行文件的名称，键入上述命令后，必须使用 GDB 命令进行调试，以下是常用的 GDN 命令：

    1. b line_number： 在给定行处创建断点，调试过程中，调试器会在断点处停止
    2. n: 运行下一行代码
    3. r: 运行程序至断点处
    4. p variable_name: 打印变量的值
    5. q: 退出调试器

1.1 Getting started with C++

1.1.2 在Ubuntu中运行C/C++

- 代码调试

```
ros@ROS-VirtualBox:~/C++_work$ gdb sum
GNU gdb (Ubuntu 8.1.1-0ubuntu1) 8.1.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sum...done.
(gdb) b 5
Breakpoint 1 at 0x912: file sum.cpp, line 5.
(gdb) r
Starting program: /home/ros/C++_work/sum

Breakpoint 1, main () at sum.cpp:7
7               int num_1 = 3;
(gdb) n
8               int num_2 = 4;
(gdb) p num_1
$1 = 3
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n)
```

1.1.3 C++中OPP概念

- OPP概述
  - OPP(Object Oriented Programming，面向对象的编程)
  - 在 C 的结构体中，用户可以将不同类型的数据(整数、浮点数和字符串)封装到一个数据类型中
  - 相似的，C++有一个增强版的结构体，在该结构体中可以定义函数
  - 该结构体称为 C++ 类，C++ 类的每个实例都被称为对象，这种编程叫做面向对象的编程

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
#include <iostream>
#include <string>
using namespace std;
struct Robot_Struct
{
        int id;
        int num_wheels;
        string robot_name;
};
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
class Robot_Class
{
public:
        int id;
        int num_wheels;
        string robot_name;
        void move_robot();
        void stop_robot();
};
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
void Robot_Class::move_robot()
{
        cout<<"Moving Robot!"<<endl;
}
void Robot_Class::stop_robot()
{
        cout<<"Stopping Robot!"<<endl;
}
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++中类和结构体的示例代码

```
int main()
{
        Robot_Struct robot_1;
        Robot_Class robot_2;
        robot_1.id=2;
        robot_1.robot_name="Mobile robot";
        robot_2.id=3;
        robot_2.robot_name="Humanoid robot";
        cout<<"ID="<<robot_1.id<<"\t"<<"Robot Name"<<robot_1.robot_name<<endl;
        cout<<"ID="<<robot_2.id<<"\t"<<"Robot Name"<<robot_2.robot_name<<endl;
        robot_2.move_robot();
        robot_2.stop_robot();
        return 0;
}
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - C++ 中类和结构体的示例代码
  - 这段代码定义了一个类和结构体。结构体的名称是 Robot_Struct，类名是 Robot_Class
  - 以下为定义的结构体。该结构体中定义了三个变量，分别是机器人编号 id、轮子的数量 num_wheels 和名称 robot_name

```
struct Robot_Struct
{
        int id;
        int num_wheels;
        string robot_name;
};
```

1.1.3 C++中OPP概念

- 类和结构体之间的区别
  - 下图左为定义的类，类中不仅可以定义变量，还可以定义函数
  - 可以在类的内部或者外部定义函数，一般在类的外部定义，以保证类的简洁性
  - 下图右为定义的类函数(成员函数)
    - ✓ 第一行第一项是返回值类型，其次是类名称，最后是用::连接的函数名，表明函数包含在类中
    - ✓ 在函数定义中，可以添加特定功能的代码，此处代码用来打印一个消息

```
class Robot_Class
{
public:
        int id;
        int num_wheels;
        string robot_name;
        void move_robot();
        void stop_robot();
};
```

```
void Robot_Class::move_robot()
{
        cout<<"Moving Robot!"<<endl;
}
void Robot_Class::stop_robot()
{
        cout<<"Stopping Robot!"<<endl;
}
```

1.1.3 C++中OPP概念

- C++的类和对象
  - 以下为创建的结构体和类的实例代码

```
Robot_Struct robot_1;
Robot_Class robot_2;
robot_1.id=2;
robot_1.robot_name="Mobile robot";
robot_2.id=3;
robot_2.robot_name="Humanoid robot";
```

  - 其中 robot_1 为结构体，robot_2 为类，类的实例称为对象
  - 因此，robot_2 是一个类对象，robot_1 是结构体的一个对象
  - 结构体和类中的变量可以使用“.”操作符进行访问
  - 如果使用结构体或者类的指针，必须使用“->”操作符访问每个变量

1.1.3 C++中OPP概念

- C++的类和对象
  - 将上述代码保存为 C++_class_test.cpp，调用 g++ 进行编译：

```
$ g++ C++_class_test.cpp -o class_struct
```

```
$ ./class_struct
```

```
ros@ROS-VirtualBox:~/C++_work$ g++ C++_class_test.cpp -o class_struct
ros@ROS-VirtualBox:~/C++_work$ ./class_struct
ID=2    Robot NmaeMobile robot
ID=3    Robot NmaeHumanoid robot
Moving Robot!
Stopping Robot!
```

1.1.3 C++中OPP概念

- 类访问修饰符
  - 上述代码中有 public: 关键字，其被称为成员访问修饰符
  - 通过设置成员访问修饰符，可以限制在类内部定义的函数的使用范围
  - 类中有3种访问修饰符：
    - public(共有)：一个公有成员可以从类以外的任何地方被访问。用户可以直接访问公共变量，不需要编写其他辅助函数
    - private(私有)：使用该修饰符号的成员变量或函数，不能从类的外部被查看或者访问。只有类和友元函数可以访问私有成员
    - protected(保护)：访问权限类似于private，不同的是其子类/派生类可以访问其成员
  - 成员修饰符可以帮助用户将变量分组，这些变量在类中保持可见或者隐藏

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用
  - Inheritance(继承)是 OPP 中的一个重要概念
  - 如有已有两个或多个类，并且在新建的类中希望使用这些已有类中的函数，可以使用 Inheritance 属性
  - 用户可以在新类中重用现有类中的函数，继承现有类的新类，称为派生类
  - 类可以通过 public、private 和 protected 来继承
    - public inheritance： 当派生自 public 基类时，基类的 public 成员成为派生类的 public 成员，并且基类的 protected 成员成为派生类的 protected 成员；基类的 private 成员不能在派生类中访问，但可以通过基类的 public 和 protected 成员函数的调用来访问

1.1.3 C++中OPP概念

- **private inheritance：当派生自 private 基类时，基类的 public 和 protected 成员成为派生类的 private 成员**
- **protected inheritance：当使用 protected 基类继承时，基类的 public 和 protected 成员成为派生类的 protected 成员**

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
#include <iostream>
#include <string>

using namespace std;
class Robot_Class
{
public:
        int id;
        int num_wheels;
        string robot_name;
        void move_robot();
        void stop_robot();
};
```

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
class Robot_Class_Derived:public Robot_Class
{
public:
        void turn_left();
        void turn_right();
};
```

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
void Robot_Class::move_robot()
{
        cout<<"Moving Robot"<<endl;
};
void Robot_Class::stop_robot()
{
        cout<<"Stopping Robot"<<endl;
};
void Robot_Class_Derived::turn_left()
{
        cout<<"Robot Turn left"<<endl;
};
void Robot_Class_Derived::turn_right()
{
        cout<<"Robot Turn right"<<endl;
};
```

1.1.3 C++中OPP概念

- C++中 Inheritance 的使用

```
int main()
{
        Robot_Class_Derived robot;
        robot.id=2;
        robot.robot_name="Mobile robot";
        cout<<"Robot ID="<<robot.id<<endl;
        cout<<"Robot Name="<<robot.robot_name<<endl;
        robot.move_robot();
        robot.stop_robot();
        robot.turn_left();
        robot.turn_right();
        return 0;
};
```

1.1.3 C++中OPP概念

- Inheritance 代码解释
  - 本程序中，创建了一个名为 Robot_Class_Derived 的新类，其派生自 Robot_Class 基类
  - 公共继承时通过关键字 public 和其后面的基类名来实现的，即在派生类名之后添加一个“：”紧接着一个 public 关键字和基类名

```
class Robot_Class_Derived:public Robot_Class
{
public:
        void turn_left();
        void turn_right();
};
```

  - 本程序中，因为选择了 public，因此继承类可以访问基类的 public 和 protected 修饰的变量和函数

1.1.3 C++中OPP概念

- Inheritance 代码解释
  - 派生类函数的定义如下：

```
void Robot_Class_Derived::turn_left()
{
        cout<<"Robot Turn left"<<endl;
}
void Robot_Class_Derived::turn_right()
{
        cout<<"Robot Turn right"<<endl;
}
```

  - 访问派生类中的变量和函数：

```
Robot_Class_Derived robot;
robot.id=2;
robot.robot_name="Mobile robot";
cout<<"Robot ID="<<robot.id<<endl;
cout<<"Robot Name="<<robot.robot_name<<endl;
robot.move_robot();
robot.stop_robot();
robot.turn_left();
robot.turn_right();
```

1.1.3 C++中OPP概念

- Inheritance 代码解释
  - 由上述代码可知，在 Robot_Class_Derived 中没有声明 id 和 robot_name 变量，而是在 Robot_Class 中定义的
  - 通过使用 inheritance 属性，可以在派生类 Robot_Class_Derived 中使用基类 Robot_Class 的变量
  - 对上述代码进行编译：

```
$ g++ C++_class_inheritance.cpp -o class_inheritance
```

```
$ ./class_inheritance
```

```
ros@ROS-VirtualBox:~/C++_work$ g++ C++_class_inheritance.cpp -o class_inheritanc
ros@ROS-VirtualBox:~/C++_work$ ./class_
class_inheritance  class_struct
ros@ROS-VirtualBox:~/C++_work$ ./class_inheritance
Robot ID=2
Robot Name=Mobile robot
Moving Robot
Stopping Robot
Robot Turn left
Robot Turn right
```

1.1.3 C++中OPP概念

- C++命名空间
  - 创建和使用命名空间的示例

```
#include <iostream>

using namespace std;
namespace robot
{
        void process(void)
        {
                cout<<"Processing by Robot"<<endl;
        }
}
```

1.1.3 C++中OPP概念

- C++命名空间
  - 创建和使用命名空间的示例

```
namespace machine
{
        void process(void)
        {
                cout<<"Processing by Machine"<<endl;
        }
}

int main()
{
        robot::process();
        machine::process();
}
```

1.1.3 C++中OPP概念

- C++命名空间
  - 创建命名空间，需要用关键词“namespace”，后面紧跟命名空间的名称
  - 从上述代码中可看出，定义了两个命名空间，命名空间中定义了相同的函数
  - 命名空间用来实现对某一特定操作的一系列函数或者类进行分组
  - 通过使用命名空间名称，后面跟“::函数名”的方式，可以访问命名空间中的成员
  - 此段代码中调用两个函数：robot::process() 和 machine::process()
  - 代码编译和运行结果如下：

```
ros@ROS-VirtualBox:~/C++_work$ g++ C++_namespace.cpp -o namespace
ros@ROS-VirtualBox:~/C++_work$ ./namespace
Processing by Rbot
Processing by Rbot
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - Linux Makefile 可以方便的利用一个指令来编译一个或多个源文件，并生成可执行文件
  - 在编写 C++ 类的时候，可以在源代码中声明和定义类，也可也使用 .h 文件和 .cpp 文件，并把 .h 文件包含到 .cpp 文件中的方式声明和定义类
  - main.cpp：将要构建的主要代码文件
  - add.h：add类的头文件，包含一个类的声明
  - add.cpp：包含类的全部定义
  - 建议：最好将类、头文件、.cpp文件用同一个名称命名。此处建立 add 类，所以头文件命名为 add.h 和 add.cpp

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 代码演示:

```
//add.h
#include <iostream>


class add
{
public:
    int compute(int num_1, int num_2);
};
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 代码演示：

```
//add.cpp
#include "add.h"


int add::compute(int a, int b)
{
    return(a+b);
}
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 代码演示：

```
//main.cpp
#include "add.h"

using namespace std;
int main()
{
        add obj;
        int result = obj.compute(3,4);
        cout<<"The result:="<<result<<endl;
        return 0;
}
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile

  - 上述代码中，main.cpp 中首先包含 add.h 头文件来访问 add 类，然后创建一个 add 类的对象，并将两个参数传递给 compute() 函数，最后打印结果

  - 使用以下命令进行编译和运行：

```
$ g++ add.cpp main.cpp -o main
```

```
$ ./main
```

```
ros@ROS-VirtualBox:~/C++_work$ g++ add.cpp main.cpp -o main
ros@ROS-VirtualBox:~/C++_work$ ./main
The result:=7
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - Linux makefile 提供了一种组织、编译多个源代码文件的方法
  - 以下为上述代码的 makefile 文件

```
CC = g++
CFLAGS = -c
SOURCES =  main.cpp add.cpp
OBJECTS = $(SOURCES:.cpp=.o)
EXECUTABLE = main
all: $(OBJECTS) $(EXECUTABLE)
$(EXECUTABLE) : $(OBJECTS)
                    $(CC) $(OBJECTS) -o $@
.cpp.o: *.h
          $(CC) $(CFLAGS) $< -o $@
clean :
          -rm -f $(OBJECTS) $(EXECUTABLE)
.PHONY: all clean
```

1.1.4 建立一个C++工程

- 建立一个Linux makefile
  - 编写makefile之后，需要使用以下命令来编译

```
$ make
```

```
ros@ROS-VirtualBox:~/C++_work$ make
g++ -c main.cpp -o main.o
g++ -c add.cpp -o add.o
g++ main.o add.o -o main
```

  - 编译以后，用以下命令来执行程序

```
$ ./main
```

```
ros@ROS-VirtualBox:~/C++_work$ ./main
The result:=7
```

ROS Summary

1. C++ basic

a) 在Ubuntu中运行C/C++



1. 安装 GCC 和 G++编译器
2. 安装和使用 GDB 调试器
3. 编写并调试 C++ 程序

b) C++中OPP概念



1. 类和结构体的定义与区别
2. C++ 中的类和对象
3. 类的访问修饰符
4. 类的 Inheritance 属性
5. 命名空间

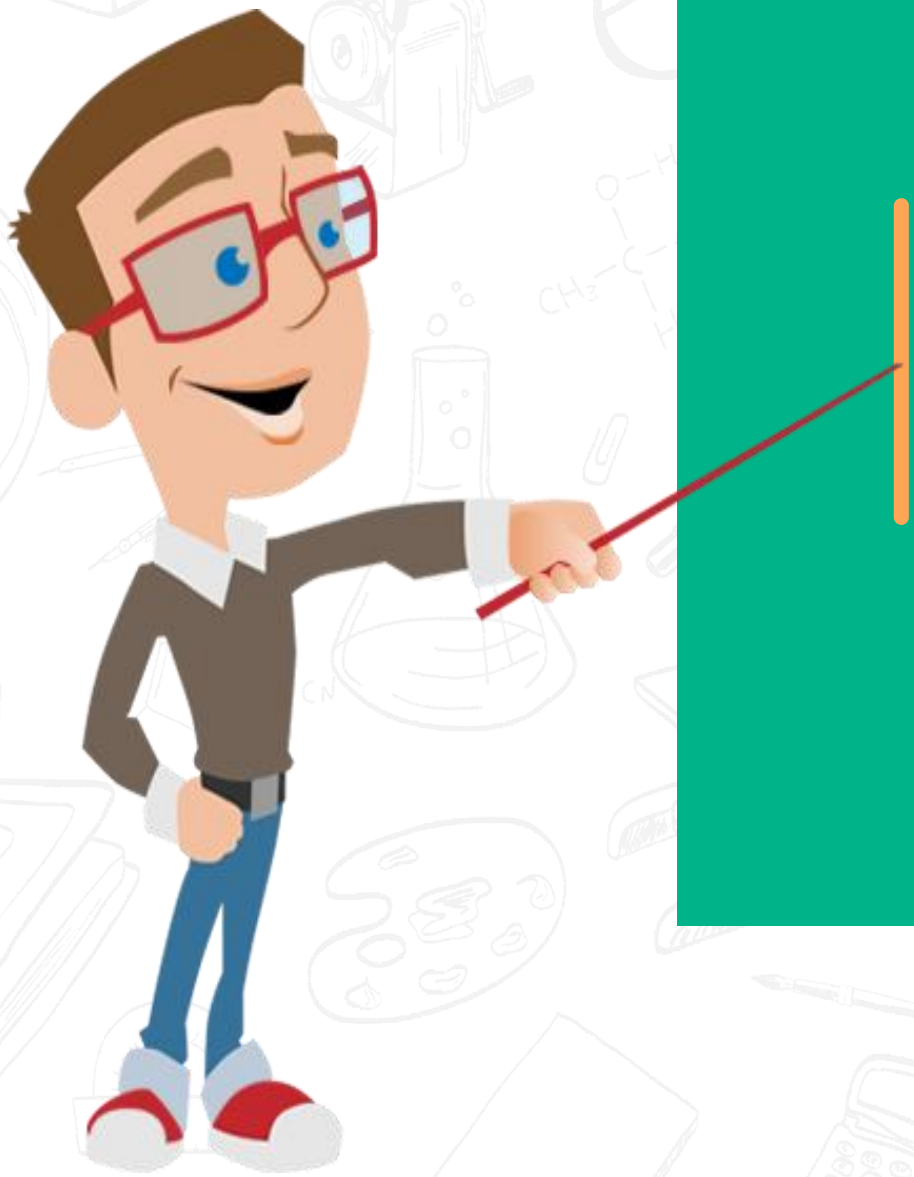
c) 建立C++工程

1. Linux makefile

Q&A

今日事今日毕

Question?



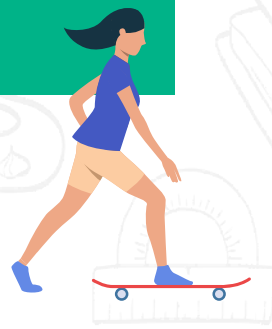
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 8-2

1. Roslaunch
2. ~
3. ~
4. ~

Objectives

1、掌握 roslaunch 编写方法

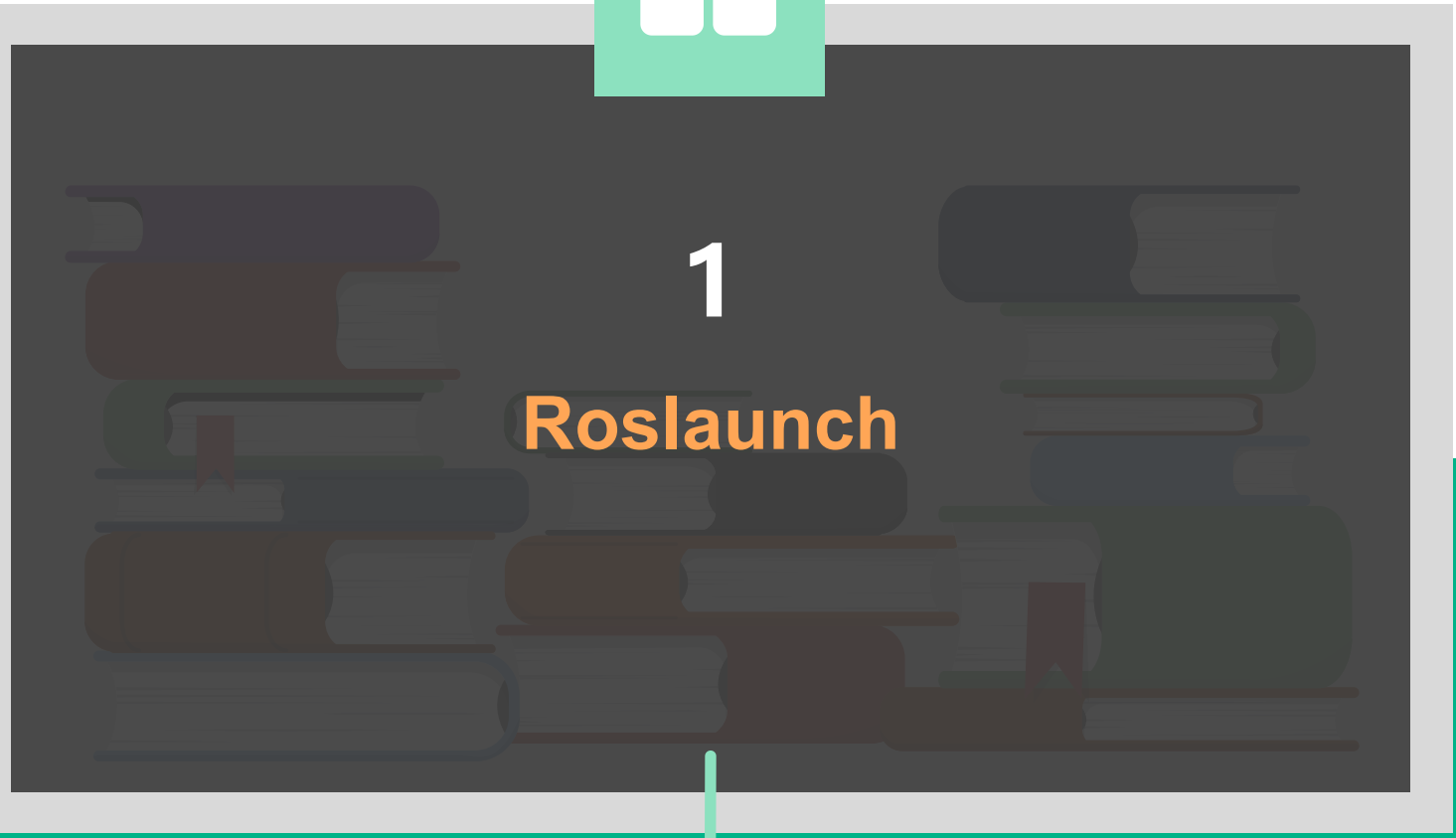
2、熟练使用 roslaunch中的 <node> 标签



Pre-assessment

1、是否编写过roslaunch文件?

2、是否用过c++编写ROS程序?



1

Roslaunch

1.1 Rosluanch format

1.1.1 Overview： http://wiki.ros.org/roslaunch/

- roslaunch 是一个工具，可以通过 SSH 在本地和远程轻松启动多个 ROS 节点，以及在参数服务器上设置参数
- roslaunch 接收一个或多个 XML 配置文件（带有 .launch 扩展名），这些文件指定要设置的参数和要启动的节点，以及它们应该在哪些机器上运行
- rosluanch 的语法：

```
$ roslaunch package_name file.launch
```

1.1 Rosluanch format

1.1.2 Roslaunch XML: http://wiki.ros.org/roslaunch/XML

- roslaunch tag reference:

<launch>	Root tag
<node>	Launch a node
<machine>	Declare a machine to use for launching
<include>	Include other roslaunch files
<remap>	Declare a name remapping
<env>	Specify an environment variable for launched nodes
<param>	Set a parameter on the Parameter Server
<rosparam>	Set ROS parameters for the launch using a rosparam file
<group>	Group enclosed elements sharing a namespace or remap
<test>	Launch a test node
<arg>	Declare an argument

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/launch

1. <launch>：roslaunch 文件的根标签，它的目的是充当其他元素的容器

   - Elements：<node>、<param>、<remap>、<machine>、<rosparam>、<include>、<env>、<test>、<arg>、<group>

2. <node> ：用于指定需要启动的 ROS 节点，用于启动和关闭节点，启动的每一个节点(node)都要有自己独一无二的名字(name)

   - <node> 标签属性：pkg、type、name、args、machine、respawn、required、ns、output、cwd、launch-prefix
   - <node> 标签中至少包含 name、pkg 和 type 属性
   - 两种使用方法：

```
<node pkg="package-name" type="node-name" name="name"  />

<node pkg="package-name" type="node-name" name="name"></node>
```

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/node

2. <node>： 指定需要启动的 ROS 节点，用于启动和关闭节点

pkg="mypackage"→Package of node, 功能包名称

type="nodetype" →Node type. There must be a corresponding executable with the same name, 节点名称

name="nodename" →Node name. NOTE: name cannot contain a namespace. Use the ns attribute instead, 节点命名

args="arg1 arg2 arg3"(optional) →Pass arguments to node, 传递节点参数

machine="machine-name"(optional, see <machine>) →Launch node on designated machine

respawn="true"(optional, default: False)→Restart the node automatically if it quits

respawn_delay="30" (optional, default 0) New in ROS indigo→If respawn is true, wait respawn_delay seconds after the node failure is detected before attempting restart

required="true"(optional)→If node dies, kill entire roslaunch

1.1 Rosluanch format

1.1.2 Roslaunch XML

2. **<node>： 指定需要启动的 ROS 节点，用于启动和关闭节点**

ns="foo"(optional)→Start the node in the 'foo' namespace，**使用命名空间**

clear_params="true|false"(optional)→Delete all parameters in the node's private namespace before launch.

output="log|screen"(optional)→If 'screen', stdout/stderr from the node will be sent to the screen. If 'log', the stdout/stderr output will be sent to a log file in $ROS_HOME/log, and stderr will continue to be sent to screen. The default is 'log'，**显示输出结果**

cwd="ROS_HOME|node"(optional)→If 'node', the working directory of the node will be set to the same directory as the node's executable. In C Turtle, the default is 'ROS_HOME'. In Box Turtle (ROS 1.0.x), the default is 'ros-root'. The use of 'ros-root' is deprecated in C Turtle.

launch-prefix="prefix arguments"(optional)→Command/arguments to prepend to node's launch arguments. This is a powerful feature that enables you to enable gdb, valgrind, xterm, nice, or other handy tools.

1.1 Rosluanch format

1.1.2 Roslaunch XML

2. <node>： 指定需要启动的 ROS 节点，用于启动和关闭节点

属性	属性作用
`name="NODE_NAME"`	为节点指派名称，这将会覆盖掉 `ros::init()` 定义的node_name
`pkg="PACKAGE_NAME"`	节点所在的包名
`type="FILE_NAME"`	执行文件的名称如果是用Python编写的就填写 `xxx.py` ，如果是cpp就写编译生成的可执行文件名
`output="screen"`	终端输出转储在当前的控制台上，而不是在日志文件中
`respawn="true"`	当roslaunch启动完所有该启动的节点之后，会监测每一个节点，保证它们正常的运行状态。对于任意节点，当它终止时，roslaunch 会将该节点重启
`required="true"`	当被此属性标记的节点终止时，roslaunch会将其他的节点一并终止。注意此属性不可以与 `respawn="true"` 一起描述同一个节点
`launch-prefix = "command-prefix"`	相当于在执行启动命令时加上一段命令前缀
`ns = "NAME_SPACE"`	这个属性可以让你在自定义的命名空间里运行节点

1.1 Rosluanch format

1.1.2 Roslaunch XML

2. <node>： 指定需要启动的 ROS 节点，用于启动和关闭节点

- 在 <node> 标签内使用以下 XML 标签：

  <env>→Set an environment variable for the node.

  <remap>→Set a remapping argument for this node.

  <rosparam>→Load a rosparam file into this node's ~/local namespace.

  <param>→Set a parameter in the node's ~/local namespace.

举例：

```
<node name="listener1" pkg="rospy_tutorials" type="listener.py" args="--test" respawn="true" />
```

说明：使用 rospy_tutorials 功能包中 listener.py 可执行文件和命令行参数 --test 启动“listener1”节点， 如果节点终止，它将自动重启

1.1 Rosluanch format

1.1.2 Roslaunch XML http://wiki.ros.org/roslaunch/XML/remap

3. <machine>：声明了一台可以运行 ROS 节点的机器。 如果在本地启动所有节点，则不需要此标签。其主要用于为远程机器声明 SSH 和 ROS 环境变量设置，也可以使用它来声明有关本地机器的信息

4. <include>：将另一个 roslaunch 文件导入当前文件。它将在文档的当前范围内导入，包括 <group> 和 <remap> 标签。包含文件中的所有内容都将被导入

5. <remap>：用于节点映射。对于有关特定 ROS 主题的消息，该消息通常只发送到一组节点，其他节点无法接收该消息。这种情况，可以进行重新映射，以便新节点在认为订阅 /different_topic 时最终订阅 /needed_topic。<remap>基本语法为：

```
<remap from="/different_topic" to="/needed_topic"/>
```

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/remap

5. <remap>：用于节点映射

remap attributes

from="original-name"→Remapped topic: name of the ROS topic that you are remapping FROM.

to="new-name"→Target name: name of the ROS topic that you are pointing the from topic TO.

- 举例：

假定一个节点它订阅了“chatter”主题，但只有一个发布到“hello”主题的节点。它们是相同的类型，此时希望将“hello”主题传输到需要“chatter”的新节点中。 可以使用此重新映射执行此操作：

```
<remap from="chatter" to="hello"/>
```

1.1 Rosluanch format

1.1.2 Roslaunch XML

5. <remap>：用于节点映射

- 注意：

1. 重映射适用于重映射之后的行。在任何重映射行之前启动的节点不受影响

2. 重映射会影响节点订阅或发布的主题。但是，通常重映射是在订阅节点上完成的，这意味着它实际上是在订阅重映射的主题。因此，任何发布节点最终都会通过发布到 remap 命令的 to 字段中指定的主题来将它们的消息发送到该节点！

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/param

6. <env>：允许在启动的节点上设置环境变量。 此标签只能在 <launch>、<include>、<node> 或 <machine> 标签的范围内使用。 当它在 <launch> 标签内使用时，<env> 标签只适用于之后声明的节点。 一些环境变量可以在 Environment Variables 中找到

7. <param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set 。可以指定文本文件、二进制文件或命令属性来设置参数的值。<param> 标签可以放在 <node> 标签内，在这种情况下，参数被视为私有参数

1.1 Rosluanch format

1.1.2 Roslaunch XML

7. **<param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set**

name="namespace/name"→Parameter name. Namespaces can be included in the parameter name, but globally specified names should be avoided.

value="value"(optional)→Defines the value of the parameter. If this attribute is omitted, **binfile, textfile or command must be specified**.

textfile="$(find pkg-name)/path/file.txt"(optional)→The contents of the file will be read and stored as a string. The file must be locally accessible, though it is strongly recommended that you use the package-relative $(find)/file.txt syntax to specify the location.

1.1 Rosluanch format

1.1.2 Roslaunch XML

7. **<param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set**

binfile="$(find pkg-name)/path/file"(optional)→The contents of the file will be read and stored as a base64-encoded XML-RPC binary object. The file must be locally accessible, though it is strongly recommended that you use the package-relative $(find)/file.txt syntax to specify the location.

command="$(find pkg-name)/exe '$(find pkg-name)/arg.txt'"(optional)→The output of the command will be read and stored as a string. It is strongly recommended that you use the package-relative $(find)/file.txt syntax to specify file arguments. You should also quote file arguments using single quotes due to XML escaping requirements.

type="str|int|double|bool|yaml"(optional)→Specifies the type of the parameter. If you don't specify the type, roslaunch will attempt to automatically determine the type. These rules are very basic:

- numbers with '.'s are floating point, integers otherwise;
- "true" and "false" are boolean (not case-sensitive).
- all other values are strings

1.1 Rosluanch format

1.1.2 Roslaunch XML

7. <param>：定义了要在参数服务器上设置的参数，相当于命令行中的 rosparam set

- 举例：

```
<param name="publish_frequency" type="double" value="10.0" />
```

配置 publish_frequency 参数的值为10

```
<param name="params_a" type="yaml" command="cat '$(find roslaunch)/test/params.yaml'" />
```

通过加载 YAML 文件配置参数

8. <rosparam>：允许使用 rosparam YAML 文件从 ROS 参数服务器加载和转储大量参数，其还可以用于删除参数。<rosparam> 标签可以放在 <node> 标签内，在这种情况下，参数被视为私有名称

1.1 Rosluanch format

1.1.2 Roslaunch XML： http://wiki.ros.org/roslaunch/XML/group

9. <group>：用于节点分组，其可以更轻松地将设置应用于一组节点。 它有一个 ns 属性，可将节点组推送到单独的命名空间中。还可以使用 <remap> 标签在整个组中应用重新映射设置

- **Attributes：**
  - **ns="namespace" (optional)**→Assign the group of nodes to the specified namespace. The namespace can be global or relative, though global namespaces are discouraged.
  - **clear_params="true|false" (optional)**→Delete all parameters in the group's namespace before launch. This feature is very dangerous and should be used with caution. ns must be specified.

1.1 Rosluanch format

1.1.2 Roslaunch XML

9. <group>：相当于顶级的 <launch> 标签，只是作为其中标签的容器。 意味着可以使用任何标签，就像通常在 <launch> 标签中使用一样

9. <group>：可包含 <node>、<param>、<remap>、<machine>、<rosparam>、<include>、<env>、<test>、<arg>标签

1.1 Rosluanch format

1.1.2 Roslaunch XML

10. <test>：在语法上类似于 <node> 标签。 它们都指定了要运行的 ROS 节点，但是 <test> 标记表明该节点实际上是要运行的测试节点

11. <arg>：用来在 launch 文件中定义参数，其允许通过指定命令行传递、通过 <include> 传递或为更高级别文件声明的值来创建更多可重用和可配置的启动文件。<arg>声明特定于单个启动文件，必须显式地将 arg 值传递给包含的文件

作业：

资料1：https://www.jianshu.com/p/13efab3f67e0

资料2：https://blog.csdn.net/github_35160620/article/details/52618271

记录学习笔记

1.1 Rosluanch format

1.1.3 Examples

- 使用 launch 文件，需要在功能包的工作空间中创建 launch 文件夹，launch 文件统一放在 launch 文件夹下

```
$ roscd  beginner_tutorials
```

```
$ mkdir launch
```

```
$ cd launch
```

```
$ gedit turtlemimic.launch
```

```
qlf@vmbox:~$ rosnode info /mimic
--------------------------------------------------------------------------------
Node [/mimic]
Publications:
 * /output/cmd_vel [geometry_msgs/Twist]
 * /rosout [rosgraph_msgs/Log]

Subscriptions:
 * /input/pose [unknown type]

Services:
 * /mimic/get_loggers
 * /mimic/set_logger_level


contacting node http://vmbox:34401/ ...
Pid: 3874
Connections:
 * topic: /rosout
    * to: /rosout
    * direction: outbound (34763 - 10.0.2.15:48972) [11]
    * transport: TCPROS
```

1.1 Rosluanch format

1.1.2 Roslaunch XML

- 举例:

```
<launch>
    <group ns="turtlesim1">
        <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
    </group>
    <group ns="turtlesim2">
        <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
    </group>
    <node pkg="turtlesim" name="mimic" type="mimic">
        <remap from="input" to="turtlesim1/turtle1"/>
        <remap from="output" to="turtlesim2/turtle1"/>
    </node>
</launch>
```

1.1 Rosluanch format

1.1.3 Examples

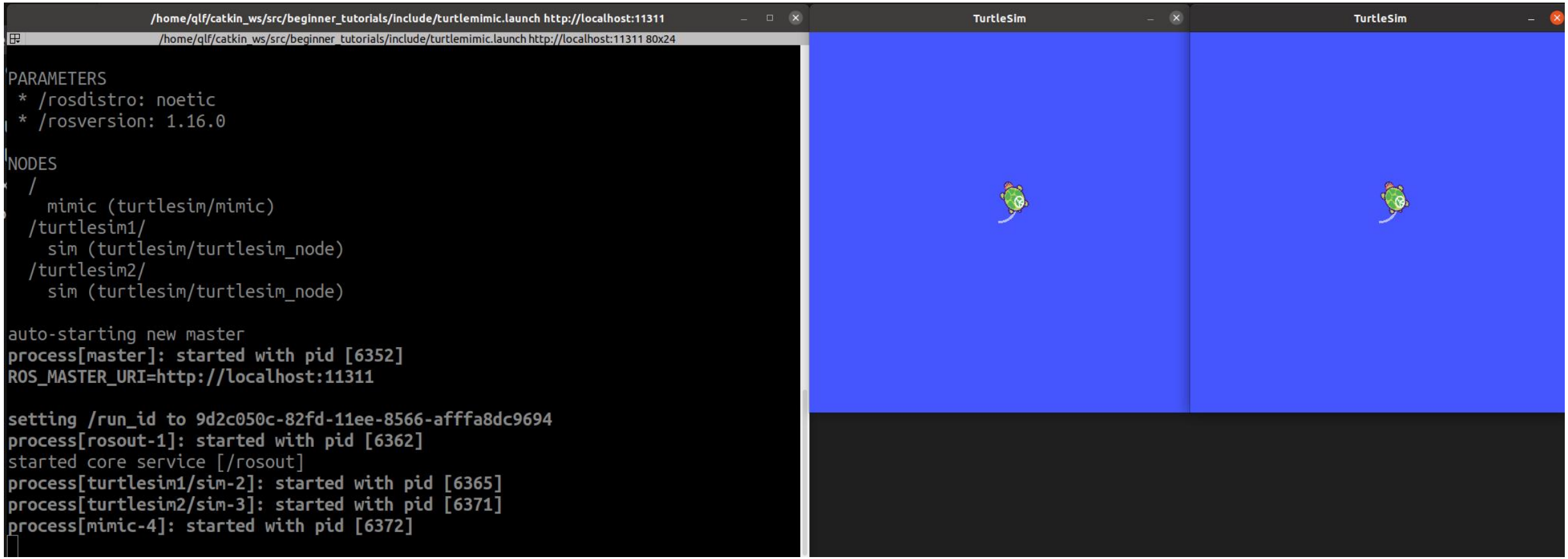
- 上述代码创建了2个 ros 节点并以不同命名空间标签区分，所以两个节点的名字都是一样的并不影响运行
- <launch> : launch 标签是其他标签的根标签，即其他标签的容器
- <group> : group 标签用于分组，” ns” 指的就是：namespace(命名空间)
- <node> : node 标签用于在终端中使用 rosrun 命令来启动一个节点
  - <node>标签包含三个属性：pkg，type 和 name，pkg 和 type 分别表示：功能包名称 和 节点名称，name 为用户为节点起的别名
- 启动launch文件：直接运行 launch 脚本文件，不用事先启动 roscore

```
$ roslaunch beginner_tutorials turtlemimic.launch
```

```
$ rostopic pub /turtlesim1/turtle1/cmd_vel geometry_msgs/Twist [1,0,0] [0,0,2]
```

1.1 Rosluanch format

1.1.3 Examples



turtlesim2 命名空间中的 turtle1 海龟的所有动作都模仿 turtlesim1命名空间中的 turtle1 海龟

1.1 Rosluanch format

1.1.3 Examples

键盘控制海归的launch文件：

```
$ gedit turtle_telecontrol.launch
```

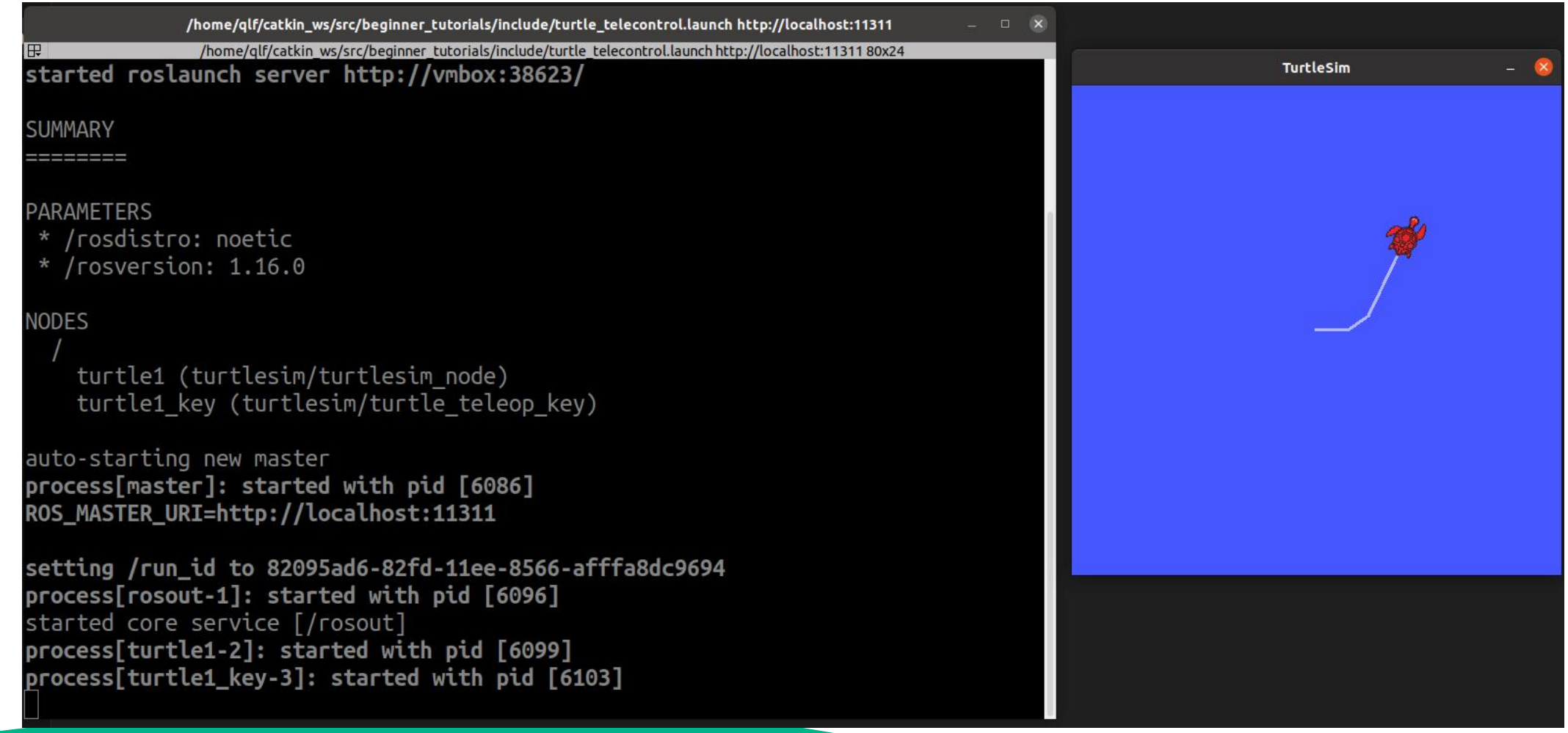
```
<launch>
    <node pkg="turtlesim" name="turtle1" type="turtlesim_node"/>
    <node pkg="turtlesim" name="turtle1_key" type="turtle_teleop_key"/>
</launch>
```

```
$ roslaunch beginner_tutorials turtle_telocontrol.launch
```

运行结果见下图：

1.1 Rosluanch format

1.1.3 Examples



ROS Summary

1. Roslaunch

a) <launch> 

b) <node> 

c) <machine>

d) <include> 

e) <remap> 

f) <env>

g) <param> 

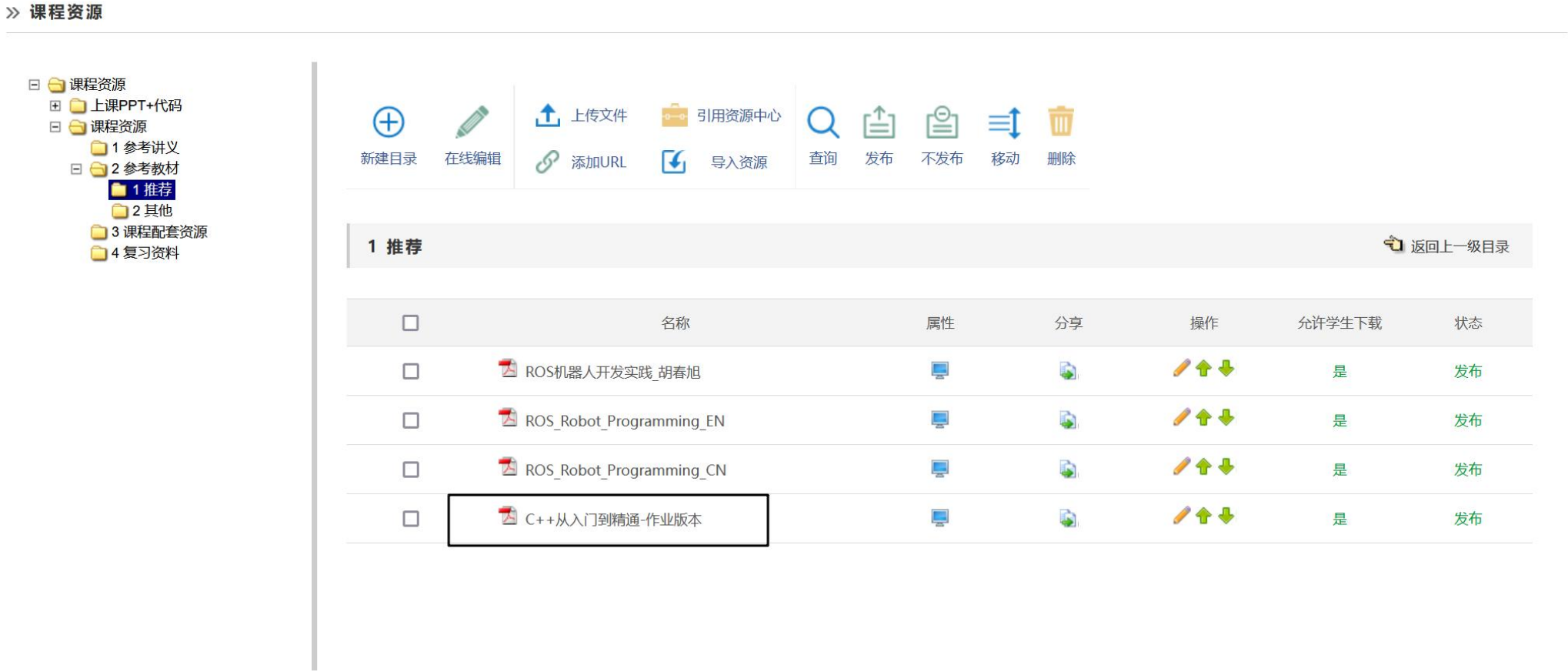
h) <rosparam>

i) <group>

j) <test>

k) <arg> 

Homework



作业：学习《C++从入门到精通-作业版本》第9章

Q&A

今日事今日毕

Question?